

# Architectural Critique

---

Spring Web Service Sandbox

Deep Architecture Review by Oracle Agent

Project: spring-web-service-sandbox

Stack: Java 25 · Spring Boot 4.0.3 · Hexagonal Architecture

Date: 9 March 2026

# 1. Architecture Assessment — Hexagonal / Ports & Adapters

---

**Verdict:** Well-applied with one notable dependency rule violation and a structural flaw in the port design.

## What Is Correct

- The package structure cleanly separates `domain`, `adapters`, and `config`. The README documents the rationale well.
- Inbound adapters ( `BookController`, `BookImportListener` ) depend on the inbound port ( `BookUseCase` ), never on the domain service directly. This is textbook.
- Outbound adapters ( `BookRepositoryAdapter`, `RabbitBookPublishAdapter` ) implement outbound ports ( `BookStore`, `BookImportPort` ). The domain never references infrastructure.
- The `Book` domain model is a plain Java record with zero framework annotations. Good domain purity.
- `open-in-view: false` in `application.yaml` — a deliberate and correct decision that prevents JPA session leakage into the web layer.

## Dependency Rule Violation

### **BookService imports Spring.**

`domain/usecase/BookService.java:6` uses `@Service` (Spring) and Lombok's `@Slf4j` and `@RequiredArgsConstructor`. In strict hexagonal architecture, the domain layer should have zero framework dependencies. The `@Service` annotation couples the domain use case implementation to the Spring DI container.

**Fix:** Move the `@Service` annotation to a `@Configuration` class in the `config` package that manually wires the bean, and replace Lombok with a hand-written constructor.

## Structural Flaw — `BookStore.save()` Accepts `CreateBookCommand`

### The outbound port takes a command rather than a domain object.

`domain/ports/out/BookStore.java:10` — The command is an inbound concept (what the outside world asks the domain to do). The outbound port should speak in domain terms. The `BookService` should construct a `Book` from the command, apply domain logic, and pass the `Book` to `BookStore.save(Book)`. Currently the domain service is a pure pass-through, making the "domain" an empty shell — the persistence adapter does the entity construction.

## Additional Observations

- `BookImportPort.enqueue()` — The name `enqueue` leaks implementation detail (queuing) into the domain vocabulary. A name like `submitForImport` would be more infrastructure-agnostic.
- **Config coupling:** Both `BookImportListener` and `RabbitBookPublishAdapter` reference `RabbitConfig.QUEUE` directly, creating compile-time coupling between inbound and outbound messaging adapters through the config class.

## 2. Design Pattern Usage

---

### Command Pattern

`CreateBookCommand` is well-applied — a clean, immutable value object that carries intent. Used consistently across the REST controller, message listener, and service.

However, the command carries no domain validation. `CreateBookRequest` validates in the web adapter, but `CreateBookCommand` accepts any strings. Messages arriving via RabbitMQ bypass all validation.

### Repository & Adapter Patterns

`BookRepositoryAdapter` wrapping `JpaBookRepository` is the correct application. The mapping between `BookEntity` (JPA) and `Book` (domain) happens here, keeping JPA concerns out of the domain.

### Over-Engineering Assessment

For a sandbox with one entity and four operations, the hexagonal structure adds ceremony (17 source files for what could be 5). But since the purpose is to learn and demonstrate the pattern, this is justified. The structure would pay off immediately when adding a second aggregate.

## 3. Code Quality

---

| Aspect       | Assessment                                                                                                                                                                    |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Record usage | Excellent. <code>Book</code> , <code>CreateBookCommand</code> , <code>CreateBookRequest</code> are all records — idiomatic modern Java.                                       |
| Lombok usage | Appropriate and restrained. Only <code>@RequiredArgsConstructor</code> , <code>@Slf4j</code> , <code>@Getter</code> . No <code>@Data</code> or <code>@Builder</code> overuse. |
| Naming       | Consistently good. Names clearly convey purpose and layer.                                                                                                                    |

---

Error  
handling

Incomplete — see below.

---

## Error Handling Gaps

- `GlobalExceptionHandler` handles `IOException` and `MethodArgumentNotValidException` but nothing else. No handler for `HttpMessageNotReadableException` (malformed JSON), `MethodArgumentTypeMismatchException` (e.g., `GET /books/abc`), or generic `Exception` fallback.
- CSV import catches no exceptions around `record.get("title")`. Missing columns cause `IllegalArgumentException` to propagate unhandled.
- `BookImportListener.onMessage()` has no error handling. A poison message causes infinite redelivery with no dead-letter queue.

## 4. Testing Strategy

---

| Layer               | Test                    | Type                         | Coverage                          |
|---------------------|-------------------------|------------------------------|-----------------------------------|
| Domain              | BookServiceTest         | Unit (Mockito)               | All 4 use case methods            |
| Web adapter         | BookControllerTest      | Slice (@WebMvcTest)          | GET, POST, validation, CSV import |
| Messaging adapter   | BookImportListenerTest  | Unit (Mockito)               | Message handling                  |
| Persistence adapter | BookRepositoryTest      | Slice + Testcontainers       | JPA save/find                     |
| Full stack          | SandboxApplicationTests | Integration + Testcontainers | Context loads only                |

### Missing Test Scenarios

1. **No test for `BookRepositoryAdapter`** — mapping logic between `BookEntity` and `Book` is untested
2. **No integration test for the RabbitMQ flow** — the full publish-consume-persist pipeline is never tested end-to-end
3. **No test for `RabbitBookPublishAdapter`**
4. **No negative test for CSV with missing columns**
5. **No test for non-numeric ID** — `GET /books/abc`

## 5. Infrastructure & DevOps

---

### Docker Compose

Clean and minimal. Four services, correct port mappings, Prometheus provisioned via volume mount.

### Security Concerns

- Hardcoded credentials in `docker-compose.yml` and `application.yml`
- Actuator exposes `env` , `beans` , `loggers` , `httpexchanges` — information-disclosure risks
- `application-local.yml` sets `show-values: always` exposing all passwords via `/actuator/env`
- No Spring Security dependency at all — all endpoints completely open
- RabbitMQ uses default `guest/guest` credentials

## Missing Infrastructure

- No Dockerfile for the application itself
- No health check definitions in Docker Compose
- `host.docker.internal` in `prometheus.yml` is macOS/Windows-only

## 6. Scalability & Production Readiness

---

### Message Handling Resilience (Critical Gap)

- **No dead-letter queue.** Poison messages cause infinite redelivery.
- **No idempotency.** Duplicate messages create duplicate books. No unique constraint on ISBN.
- **No transactional boundary** around `bookUseCase.create()` in the listener.
- **Synchronous enqueueing** — 10K CSV rows = 10K individual `convertAndSend()` calls in one HTTP request. No batching, no publisher confirms.

### Database Concerns

- No index on `isbn` — lookup by ISBN would be slow on a large table
- No unique constraint on `isbn`
- No `created_at` / `updated_at` audit columns
- `findAll()` with no pagination loads entire table into memory

### API Design Gaps

- No pagination on `GET /books`
- No `DELETE` or `PUT` endpoints
- `POST /books/import` returns empty 202 — no job ID, no progress tracking
- `GET /books` returns domain `Book` records directly as JSON — API shape coupled to domain model

## 7. Strengths

---

1. **Clean hexagonal structure.** The separation is genuine, not superficial. The domain has zero JPA annotations, zero Spring annotations on its model classes.
2. **Idiomatic modern Java.** Records, `var`, text blocks, `Optional` — reads like Java 21+ code should.



3. **Lazy CSV parsing.** The Apache Commons CSV parser is lazy, meaning a 10MB file won't be loaded into memory.
4. **Well-structured test pyramid.** Unit, slice, and integration tests at appropriate levels.
5. **Observability built in from the start.** Prometheus, Grafana, @Timed , actuator.
6. **open-in-view: false .** Prevents one of the most common Spring Boot anti-patterns.
7. **Excellent README.** Better than most production projects.

## 8. Weaknesses & Recommendations (Prioritized)

---

### Priority 1 — Correctness Issues

| ID | Issue                                                       | Recommendation                                                                                         |
|----|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| W1 | No dead-letter queue or error handling on RabbitMQ consumer | Add DLQ binding in <code>RabbitConfig</code> , configure retry with backoff, add error handler         |
| W2 | No unique constraint on ISBN                                | Add unique index via Flyway migration, handle <code>DataIntegrityViolationException</code>             |
| W3 | Validation lives in adapter, not domain                     | Create self-validating <code>CreateBookCommand</code> or validate in <code>BookService.create()</code> |

### Priority 2 — Design Improvements

| ID | Issue                                                                  | Recommendation                                                            |
|----|------------------------------------------------------------------------|---------------------------------------------------------------------------|
| W4 | <code>BookStore.save()</code> accepts command instead of domain object | Service should construct <code>Book</code> from command and pass to store |
| W5 | No pagination on <code>GET /books</code>                               | Add <code>Pageable</code> support to <code>BookStore</code>               |
| W6 | <code>@Service</code> on domain use case                               | Wire via <code>@Bean</code> method in configuration class                 |
| W7 | No <code>BookRepositoryAdapter</code> test                             | Add unit test with mocked <code>JpaBookRepository</code>                  |

### Priority 3 — Operational Hardening

| ID | Issue                           | Recommendation                                  |
|----|---------------------------------|-------------------------------------------------|
| W8 | CSV import provides no feedback | Return enqueued/skipped counts or a job ID      |
| W9 | Actuator endpoints wide open    | Add Spring Security or separate management port |

|     |                           |                            |
|-----|---------------------------|----------------------------|
| W10 | No application Dockerfile | Add multi-stage Dockerfile |
|-----|---------------------------|----------------------------|

## Priority 4 — Minor Refinements

| ID  | Issue                                                   | Recommendation                               |
|-----|---------------------------------------------------------|----------------------------------------------|
| W11 | <code>enqueue()</code> leaks implementation into domain | Rename to <code>submitForImport()</code>     |
| W12 | Location URI via string concatenation                   | Use <code>ServletUriComponentsBuilder</code> |
| W13 | <code>host.docker.internal</code> not portable          | Document or add network alias                |
| W14 | Direct queue publish bypasses exchange model            | Use explicit exchange and routing key        |